

Petra: Enhancing Efficiency in Privacy-Preserving Three-Party Relational Analytics for Private Data

Ziwei Peng^{ID}, Graduate Student Member, IEEE, Kaiping Xue^{ID}, Senior Member, IEEE,
Jingcheng Zhao^{ID}, Graduate Student Member, IEEE, Jinjiang Yang^{ID}, and Yingjie Xue^{ID}, Member, IEEE

Abstract—As organizations increasingly rely on collaborative relational analytics over private databases, secure multi-party computation (MPC) has emerged as a key solution to process private data. While numerous schemes optimize many MPC-based algorithms for complex relational queries, the basic MPC operations underlying relational computation, such as comparisons, still cost high communication overhead and latency. This imposes a significant bottleneck. To further enhance practicality and efficiency, we aim to reduce the overhead of using MPC, enabling more efficient computation of relational operators and query execution. We propose Petra, a three-party model solution for privacy-preserving relational analytics. Petra introduces new primitives of efficient equality and inequality operations designed for databases. These primitives leverage a pseudorandom transformation and a “shuffle-and-compare” working pattern. Then, by composing these primitives and designing relational operator protocols including SELECT, aggregation, Join, and Group-by, Petra enables efficient execution of SQL operators. Importantly, Petra provides different security guarantees compared to typical MPC protocols, as it allows a limited form of leakage to one of the computing parties. This trade-off is central to Petra’s design. Experimental results demonstrate that Petra significantly improves query performance compared to existing approaches and scales to complex analytics over large databases.

Index Terms—Privacy preservation, relational analytics, multi-party computation, secret sharing.

I. INTRODUCTION

RELATIONAL analytics, which involve SQL-like queries [1], [2], [3], are among the most popular applications of data analysis, particularly in databases. Relational data is often organized into tables with rows and columns, where each table represents an entity or object. The columns define specific attributes of the tuples within

the entity. Relational analytics suffice to fulfill most of the functional demands of database services, with capabilities of performing aggregations, filtering data based on composed conditions, and bridging links between multiple tables.

As data is increasingly distributed across and held by multiple owners, collaborative analytics is in rising demand. For example, a government administration possesses a name list and seeks to assess credit ratings for a partition of clients across two commercial banks. This requires joint analysis of distributed data spanning three institutions and diverse records such as account profiles, transaction histories and income. Similarly, multiple hospitals may seek to enable outsourced analysis of their combined medical records in response to requests from research institutions. While such collaborations offer significant value, they also raise privacy concerns, especially regarding personal or proprietary information [4], [5], [6]. Owners cannot directly share their data, as doing so would expose sensitive details to unauthorized parties. Therefore, there is a critical need for data privacy preservation in collaborative relational analytics.

One common approach to addressing the privacy requirements of collaborative analytics is Secure Multi-Party Computation (MPC). MPC enables multiple parties to securely compute a specific function f over their private inputs. The computing parties learn no information about the inputs and outputs of other parties. It allows this analysis to proceed without revealing the owners’ data and the analytics result.

Many works proposed various cryptographic techniques and secure multi-party computation protocols [7], [8], [9], [10] for collaborative relational analytics. Since direct use of MPC incurs high costs and poor performance, prior works mainly focused on minimizing the usage of MPC and the associated overhead. Early approaches reduced MPC overhead by leveraging data owners’ local computations. Their underlying cryptographic tools are generic MPC techniques such as oblivious RAM [11], [12] or garbled circuits [13], [14], [15]. Recent works [16], [17], [18], [19] employed secret sharing and designed algorithms to reduce the computational complexity of secure operators, such as join and group-by functions.

Despite advancements in developing low-complexity algorithms, current implementations remain inefficient. For example, executing a single query with an equality comparison on one million records take tens of seconds. This inefficiency arises from the fact that secure relational operators and queries rely heavily on repeated basic operations—such as comparisons and

Received 8 October 2025; revised 14 May 2026; accepted 28 May 2026. Date of publication 8 June 2026; date of current version 1 September 2026. This work was supported in part by the Innovation Program for Quantum Science and Technology under Grant 2021ZD0301301, in part by the National Natural Science Foundation of China (NSFC) under Grant 62372425, and in part by the Youth Innovation Promotion Association of Chinese Academy of Sciences (CAS) under Grant Y202093. (Corresponding author: Kaiping Xue.)

Ziwei Peng, Jingcheng Zhao, Jinjiang Yang, and Yingjie Xue are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, China.

Kaiping Xue is with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, China, and also with Hefei National Laboratory, Hefei 230088, China (e-mail: kpxue@ustc.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3701010>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3701010

multiplications—that are realized by MPC techniques. When these operations are applied to large relational databases using MPC, the resulting computations become resource-intensive and incur significant cryptographic overhead. To improve the practicality of privacy-preserving relational analytics for real-time applications, we explore alternatives to secure relational operators. Our research is conducted in two stages, each addressing specific challenges.

First, we identify that the inefficiency of basic computations required for relational operators—specifically, equality comparison, logical AND gate, and inequality comparison—serves as a key bottleneck in query performance. The operators of relational analytics typically involves simple arithmetic operations (e.g., addition) and non-arithmetic operations (e.g., comparisons and group-by). While simple arithmetic operations can often be performed efficiently in many MPC techniques, these non-arithmetic operations require interactive MPC protocols, resulting in high communication overhead and increased latency. Furthermore, they are frequently used by relational operators and their compositions, which further amplifies the communication time costs. To address this, we propose new lightweight primitives for two comparisons and the logical AND gate, by leveraging two-party secret sharing and a pseudorandom transformation combined with shuffling. This introduces a novel “shuffle-and-compare” working pattern specifically designed for computation on databases. Unlike traditional cryptographic protocols, our primitives following this pattern require minimal computation and communication, thus achieving significant performance gains.

Second, we leverage our new primitives to support relational operators and their compositions, enabling a system with powerful functionality for relational analytics. While basic primitives cannot directly handle multiple operands required by operators like aggregation, join, and group-by, we integrate an auxiliary operand with our primitives, which allows us to construct a multiplication-like protocol, known as Multiplexer, without introducing additional communication rounds. With Multiplexer, we can compute SELECT, MIN/MAX and aggregation with filters. We further develop protocols for other operators such as join, semi-join and group-by. These operators can be composed to efficiently support relational queries as the underlying primitives are efficient.

To summarize, our main contributions are as follows.

- We introduce a new “shuffle-and-compare” working pattern for reducing the overhead of directly using cryptographic techniques on databases. This presents a new privacy-preserving protocol for equality comparison in three-party computation, and it requires only one pseudorandom number amortized to each element. Using similar ideas, we propose two more primitives for computing AND gate and inequality.
- We further provide protocols for relational operators based on our new primitives and realize a Privacy-preserving and Efficient Three-party system for Relational Analytics called Petra. We develop a multiplexer protocol carefully linked with Petra’s primitives, and based on it we design protocols that offer rich functionalities, including

aggregation with selecting filters, logical composition, join, and group-by.

- We implement Petra and conduct experiments to evaluate its query processing capabilities and efficiency. Results demonstrate that Petra outperforms prior works and makes relational analytics more accessible for practice. For a TPC-H query, Petra completes in one second and achieves a 20-fold improvement over prior works.

II. RELATED WORK

A. Relational Data Analytics

There are numerous works in the literature on relational data analytics based on secure multi-party computation (MPC) techniques. Early works [11], [12], [13], [14], [15] focus on a distinct setting of private database federation, where mutually untrusted data owners act as computing parties and run public queries on the combined data. These works leverage generic MPC techniques such as oblivious RAM (ORAM) and garbled circuits (GC), which offer universality for various analysis tasks but incur significant computation overhead and high latency. Optimizations in these works include computing over local databases or allowing extracting the valid data rows after applying multiple filters. Our system adopts a different setting, where the computing parties perform secure outsourced computation without accessing the plaintext data, and the requester reconstructs the secure outputs from the parties to obtain the final results.

Recent works advanced secure multi-party data analytics using secret sharing methods. Waldo [20] combines function secret sharing and replicated secret sharing to enable efficient aggregation queries with multiple predicates over private databases. The query functionality of Waldo is designed for time-series databases, so relational queries like group-by, join, and sort are not supported. SECRECY [16] employs only replicated secret sharing [9] with innovative cost-based query optimizations for composing various relational operators, significantly enhancing performance compared to generic techniques like ORAM and GC [21]. TVA [22] also uses replicated secret sharing to offer a comprehensive toolbox for secure time-series analytics. However, though the algorithm complexity of secure operators and queries is optimized, one underlying bottleneck is that the underlying replicated secret sharing requires many communication rounds for computing comparison predicates and re-sharing after multiplication. In Petra, we try to reduce the overhead associated with MPC. We propose new primitives using two-party secret sharing designed for operations across the entire database, fundamentally improving efficiency.

Another line of related work focuses on using MPC techniques to design standalone relational operators: join by unique keys [17], [18], group-by [19], join-aggregation [23], set computation [24] and collection-involved aggregations [25]. Scape [18] provides an algorithm for joining by sorted (unique) keys using oblivious merge. [19] provides algorithms for joining by unique keys and aggregation by groups using oblivious sort [26], which requires shuffling the entire database. It is interesting future work

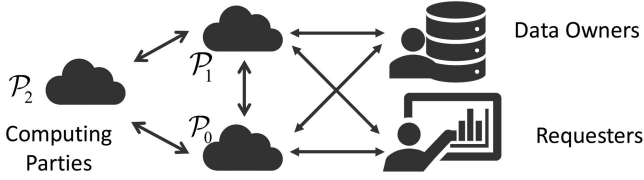


Fig. 1. System Model.

to integrate these schemes of single operators with special cases into a cohesive system. In Petra, we give a general construction for relational operators, operator composition and query execution, based on our newly developed primitives different from typical MPC closed boxes of basic operations.

B. Encrypted Databases

Encrypted databases [27], [28] support relational queries on encrypted data by selectively using specialized encryption schemes or pre-constructing encrypted indices. Existing systems, such as CryptDB [29], SPX [30], Arx [31], [32] and [33], employ these methods. The supporting functionalities include range query and group-aggregation query. These approaches differ from multi-party collaborative analytics schemes, as in their client-server querying setting the encrypted indices as well as encrypted data are constructed and uploaded by the data owner.

III. PRELIMINARIES

A. System Model

Petra's system model involves three roles: data owners, computing parties, and query requesters [19], [22]. Data owners have private data and do not want to reveal it in public. Query requesters issue SQL queries to request analytics results on the combined data. The computing parties are authorized by the owners and perform secure computations on their data. Fig. 1 illustrates this system.

For the computing parties, Petra uses an unbalanced three-party computation setting, involving three parties: $\mathcal{P}_0$, $\mathcal{P}_1$, and $\mathcal{P}_2$. Data owners upload their data to $\mathcal{P}_0$ and $\mathcal{P}_1$ (using secret sharing). An unbalanced setting means that $\mathcal{P}_0$ and $\mathcal{P}_1$ have secret sharings as their private inputs, while $\mathcal{P}_2$ has no input. When the requester issues a query to the two parties, they perform privacy-preserving relational analytics protocols together with $\mathcal{P}_2$. The requester receives the outputs from $\mathcal{P}_0$ and $\mathcal{P}_1$ to reconstruct the plaintext results.

B. Security Model

We adhere to an honest-but-curious threat model in an honest-majority setting, meaning the three parties follow the protocol as specified, and at most one party may be corrupted by an honest-but-curious adversary. It is assumed that there is no collusion between the three parties. This three-party security model is consistent with many prior works [16], [22], [34]. In this model, Petra ensures that no single party can compromise the inputs,

intermediate computations, or outputs alone. The non-colluding assumption is practical in many real-world scenarios, such as when computing parties are independent cloud service providers or distinct organizations with no incentive to collude. Data owners and requesters are assumed to act honestly, following the protocol and granting explicit authorization for data usage or queries.

C. Design Goals

We aim to enable efficient and privacy-preserving relational analytics. To this end, we outline the following goals:

- 1) *Data privacy-preservation*: Data owners upload their private data in a secure way so that the computing parties cannot access the plaintext data. During the computation, the computing parties learn no information about which data is accessed. The query is public to all parties.
- 2) *Functionality of relational analytics*: The basic query types of relational analytics should be covered, including equality and inequality predicates, SELECT, logical composition, aggregation, and complex predicates such as Join and Group-by.
- 3) *High efficiency*: The overall efficiency of the protocols is essential. Communication and computation should be efficient, avoiding heavy overhead associated with cryptographic techniques such as garbled circuits.

D. Secret Sharing and Notations

Petra uses the two-party additive secret sharing scheme. Data owners upload their data as secret shares and distribute them between $\mathcal{P}_0$ and $\mathcal{P}_1$. Two types of additive sharing are used in Petra. The first type is boolean sharing. For an l -bit number x , its boolean shares are two random l -bit numbers $\llbracket x \rrbracket^0$ and $\llbracket x \rrbracket^1$ such that $x = \llbracket x \rrbracket^0 \oplus \llbracket x \rrbracket^1$, where $\oplus$ denotes the bitwise XOR. The shares support XOR locally $\llbracket x \rrbracket \oplus \llbracket y \rrbracket = \llbracket x \oplus y \rrbracket$. The bitwise AND of two numbers is denoted as xy (for shares, $\llbracket x \rrbracket \llbracket y \rrbracket$). In particular, when x and y are single bits, we denote $x \wedge y$ for the AND of two bits and $\neg x$ for the NOT of one bit. Local NOT of two-party sharing follows a property: $\neg \llbracket x \rrbracket^0 \oplus \neg \llbracket x \rrbracket^1 = x$, $\neg \llbracket x \rrbracket^0 \oplus \llbracket x \rrbracket^1 = \neg x$. In addition to boolean sharing, the other type used in Petra is arithmetic secret sharing. For an integer x from $\mathbb{F}_p$, its arithmetic shares are two numbers $\langle\langle x \rangle\rangle^0$ and $\langle\langle x \rangle\rangle^1$ randomly sampled from $\mathbb{F}_p$ such that $x = \langle\langle x \rangle\rangle^0 + \langle\langle x \rangle\rangle^1 \bmod p$.

Vectorization: Petra runs on databases, so all operators are designed to work on a whole column of a relation, and all the functions are component-wise operations on vectors. We denote a vector $(x_1, x_2, \dots, x_n)$ as $\vec{x} := (x_i)_{i \in [1, n]}$, where $[1, n]$ is shorthand for $\{1, 2, 3, \dots, n\}$. Similarly, the share of a vector is denoted as $\llbracket \vec{x} \rrbracket := (\llbracket x_i \rrbracket)_{i \in [1, n]}$, and the notation $\llbracket \vec{x} \rrbracket \llbracket \vec{y} \rrbracket$ represents a component-wise operation of the bitwise AND.

Relation Notations: A relation is organized as tuples with values of attributes $\{(v_{i1}, v_{i2}, \dots, v_{in})\}_{i \in [1, n]}$. Given a relation R , we denote r_i as its i -th tuple, $r_i[a_k] = v_{ik}$ as the value of the k -th attribute of r_i , and $r[a_k]$ as the array containing all values of the attribute a_k . n denotes the relation size $|R|$.

Pseudorandom function (PRF) and Permutation: $\mathcal{P}_0$ and $\mathcal{P}_1$ have pre-negotiated pseudorandom seeds. Using their private

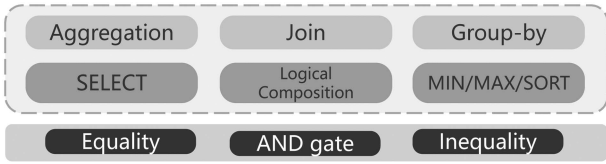


Fig. 2. The Structure of Petra.

seeds, they can generate a pseudorandom function (PRF) and pseudorandom permutations π . In this paper, we denote a PRF as a keyed one-way function $F_K(x) : \{0, 1\}^l \rightarrow \{0, 1\}^l$, where the key K is held by Parties 0 and 1. For security guarantees, the key is derived from the seed and is updated for each use.

IV. PETRA SCHEME

A. Scheme Overview

Fig. 2 illustrates the structure of Petra. At the base layer, we provide new protocols for three primitives: Equality, AND gate and Inequality in Section IV-B, which serve as foundations for Petra system. The primitives leverage two-party secret sharing that is distributed among two parties $\mathcal{P}_0$ and $\mathcal{P}_1$, while fully utilizing the assistance of the third computing party $\mathcal{P}_2$. This design reduces communication rounds and time cost compared to typical MPC techniques. The first primitive achieves equality comparisons over a bulk of secret shares. The approach involves converting two-party shares into pseudorandom numbers using a PRF. Upon examining the new Equality primitive, we identified that it supports any AND composition of equality by simply concatenating inputs. This mechanism, functioning as an AND gate, is defined as our second primitive. Furthermore, using the AND gate for boolean circuits, we can easily compute inequality over boolean shares, leading to our third primitive: Inequality.

At the upper layer of Petra, we provide protocols for various relational operators in Section IV-C. We first introduce another privacy-preserving protocol, Multiplexer [16], [35], adapted from our basic primitives, and use it in aggregation with SELECT filters and many other operators. Prior works [16], [20], [22] leverage secure multiplication for this operator, which requires an extra communication round. In contrast, we combine the multiplexer with our primitives, saving one communication round. By carefully placing our basic primitives alongside the multiplexer, we provide efficient protocols for MIN/MAX aggregation and Sort. Then, for logical composition, we use our AND gate primitive to efficiently combine multiple operators in complex queries. We also deliver protocols for Join and Group-by. We further propose optimizations for operator compositions in Section IV-D.

Petra allows a limited form of leakage revealed to $\mathcal{P}_2$: the output results of equality comparisons and AND compositions. This leakage is reduced by shuffling the data before sending it to $\mathcal{P}_2$, preventing it from learning which specific records satisfy the conditions. We note that the effectiveness of shuffling as a linkage protection mechanism degrades when the data batch size is small. For a batch of size n , there are $n!$ possible permutations. If n is too small, the space of possible mappings is limited,

Equality ($\mathcal{F}_{eq}$):

This primitive performs component-wise equality comparison on n -element secret vectors $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$, and outputs the shares of the comparison result as $\llbracket b \rrbracket$. If $x_i = y_i$, the i -th element of the result b_i is 1; otherwise, $b_i = 0$.

AND gate ($\mathcal{F}_{AND}$):

This primitive performs a component-wise boolean AND computation on two secret bit vectors $\llbracket p \rrbracket$ and $\llbracket q \rrbracket$ (length n). It outputs a secret vector $\llbracket r \rrbracket$, where $r_i = p_i \wedge q_i$.

Inequality ($\mathcal{F}_{ineq}$):

This primitive performs component-wise comparison on n -element secret vectors $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$, and outputs the shares of the comparison result as $\llbracket b \rrbracket$. If $x_i > y_i$, the i -th element of the result b_i is 1; otherwise, $b_i = 0$.

Fig. 3. Primitive Definitions of Petra.

making it easier for $\mathcal{P}_2$ to reason about potential linkages through auxiliary information, thereby weakening privacy. As a general guideline, we recommend batches of at least several dozen items (e.g., $n \geq 40$) to ensure the permutation space is large enough.

B. Privacy-Preserving Primitives Over Databases

This is the starting point of Petra. We introduce three primitives: equality comparison of boolean or arithmetic secret shares, AND gate and inequality comparison of boolean shares. They are performed on a column of the entire databases. The definitions are listed in Fig. 3.

1) *Equality*: The equality of two data columns (vectors) is a component-wise comparison of each element. For each component, we consider the equality expression:

$$\llbracket x \rrbracket^0 \oplus \llbracket x \rrbracket^1 = \llbracket y \rrbracket^0 \oplus \llbracket y \rrbracket^1.$$

By shifting the items to the same party's shares, we obtain:

$$F_K(\llbracket x \rrbracket^0 \oplus \llbracket y \rrbracket^0) = F_K(\llbracket x \rrbracket^1 \oplus \llbracket y \rrbracket^1),$$

Fig. 4 presents a detailed textual description of the protocol design. $\mathcal{P}_0$ and $\mathcal{P}_1$ apply a pseudorandom conversion to both sides (the PRF key is updated for each element), and $\mathcal{P}_2$ compares the pseudorandom numbers. If the results match, the equality holds, and $\mathcal{P}_2$ sets the bit b to one. (This approach applies similarly to arithmetic shares.) To prevent $\mathcal{P}_2$ from knowing which row matches the query value, $\mathcal{P}_0$ and $\mathcal{P}_1$ shuffle the PRF results using a permutation generated from their seeds, thereby breaking the direct link between the comparison results and the original row order. In the final step, $\mathcal{P}_0$ and $\mathcal{P}_1$ receive from $\mathcal{P}_2$ the shares of the equality bits consistent with the shuffled data. They can then obtain the shares of the correct bits via reshuffling.

Working Pattern: The Equality primitive outlines a unique working pattern that Petra's primitives follow. As Fig. 5 shows, $\mathcal{P}_0$ and $\mathcal{P}_1$ convert the two-party secret shares into pseudorandom numbers using PRF, locally shuffle them, and then $\mathcal{P}_2$ compares the two sets of PRF results one-by-one. After $\mathcal{P}_2$ shares the intermediate output, $\mathcal{P}_0$ and $\mathcal{P}_1$ reshuffle the set, and perform certain computations to obtain the secret shares of the outputs.

Input: $\mathcal{P}_0$ and $\mathcal{P}_1$ hold sharings of two n -element secret vectors $\llbracket \vec{x} \rrbracket$ and $\llbracket \vec{y} \rrbracket$. The elements are l -bits long.

Output: $\mathcal{P}_0$ and $\mathcal{P}_1$ obtain a boolean share $\llbracket \vec{b} \rrbracket$, where the i -th element b_i indicates the equality of two numbers x_i and y_i . $\mathcal{P}_2$ obtains a count number, i.e., $\sum_{i=0}^n \vec{b}_i$.

Protocol:

1. $\mathcal{P}_0$ and $\mathcal{P}_1$ do XOR locally and compute PRF for each element of their input vectors: $f_i = F_{K_i}(\llbracket x_i \rrbracket \oplus \llbracket y_i \rrbracket)$, $i \in [1, n]$, yielding their own list $(f_1, f_2, \dots, f_n)$. The two lists are denoted as S^0 and S^1 .
2. $\mathcal{P}_0$ and $\mathcal{P}_1$ generate a pseudorandom permutation π , shuffle the list $S^0 \leftarrow \pi(S^0)$, $S^1 \leftarrow \pi(S^1)$ and send to $\mathcal{P}_2$.
3. For $i \in [1, n]$, $\mathcal{P}_2$ performs a component-wise comparison between the two lists S^0 and S^1 to generate a binary vector $\vec{b}$ as follows. If two pseudorandom numbers match, $b_i \leftarrow 1$. Otherwise, $b_i \leftarrow 0$.
4. $\mathcal{P}_2$ splits the result as $\llbracket \vec{b} \rrbracket^0$ and $\llbracket \vec{b} \rrbracket^1$, and sends $\llbracket \vec{b} \rrbracket^0$ to $\mathcal{P}_0$ and $\llbracket \vec{b} \rrbracket^1$ to $\mathcal{P}_1$.
5. $\mathcal{P}_0$ and $\mathcal{P}_1$ reshuffle and return $\llbracket \vec{b} \rrbracket \leftarrow \pi^{-1}(\llbracket \vec{b} \rrbracket)$.

Fig. 4. Protocol of Primitive Equality.



Fig. 5. An Illustration of Equality and Working Pattern.

2) *AND Gate*: The primitive of equality comparison has the merit of being able to combine any number of equalities together. This provides a mechanism that can be used in the AND gate of a circuit, which we define as our second primitive. The AND gates can be defined as follows:

$$p \wedge q \Leftrightarrow (p = 1?) \wedge (q = 1?),$$

$$p_1 \wedge p_2 \wedge \dots \wedge p_k \Leftrightarrow (p_1 = 1?) \wedge \dots \wedge (p_k = 1?),$$

$$p_1 \wedge \dots \wedge p_{k-1} \wedge \neg p_k \Leftrightarrow (p_1 = 1?) \wedge \dots \wedge (p_{k-1} = 1?) \wedge (p_k = 0?),$$

where the values p are shared between two parties as $\llbracket p \rrbracket$.

The equality compared with a boolean constant (1 or 0) is the same as what we introduced previously: $\llbracket p_i \rrbracket^0 = \llbracket p_i \rrbracket^1 \oplus 1 = \neg \llbracket p_i \rrbracket^1$, $\llbracket p_i \rrbracket^0 = \llbracket p_i \rrbracket^1 \oplus 0 = \llbracket p_i \rrbracket^1$. Thus, we can compute multiple AND gates by concatenating each bit together before computing the PRF. For example, to compute $p_1 \wedge p_2 \wedge \dots \wedge p_{k-1} \wedge p_k$, $\mathcal{P}_0$ uses

$$F_K(\llbracket p_1 \rrbracket^0 || \llbracket p_2 \rrbracket^0 || \dots || \llbracket p_k \rrbracket^0)$$

($||$ denotes concatenation), and $\mathcal{P}_1$ uses

$$F_K(\neg \llbracket p_1 \rrbracket^1 || \neg \llbracket p_2 \rrbracket^1 || \dots || \neg \llbracket p_k \rrbracket^1).$$

The arithmetic sharing can also be concatenated together using the AND gate primitive. For example, $(p) \text{AND}(x = q)$ can be computed by matching $F_K(\llbracket p \rrbracket^0 || \llbracket x \rrbracket^0)$ and $F_K(\neg \llbracket p \rrbracket^1 || q - \llbracket x \rrbracket^1)$. Note that following Petra's working pattern, shuffling and reshuffling are necessary to protect data privacy.

3) *Inequality*: The inequality comparison primitive for two vectors is a component-wise comparison and it only supports boolean shares. The comparison boolean circuit consists of a series of AND gates, allowing us to utilize the two aforementioned primitives. For simplicity, next we consider the greater-than operator. Given two numbers x and y of bit-length l (in binary form, $(x_l x_{l-1} \dots x_1)_2, (y_l y_{l-1} \dots y_1)_2$, respectively) shared between two parties as $\llbracket x \rrbracket, \llbracket y \rrbracket$, the circuit is:

$$\begin{aligned} & (x_l \oplus y_l) \wedge x_l \\ & \oplus \neg(x_l \oplus y_l) \wedge (x_{l-1} \oplus y_{l-1}) \wedge x_{l-1} \\ & \oplus \neg(x_l \oplus y_l) \wedge \neg(x_{l-1} \oplus y_{l-1}) \wedge (x_{l-2} \oplus y_{l-2}) \wedge x_{l-2} \\ & \oplus \dots \\ & \oplus \neg(x_l \oplus y_l) \wedge \dots \wedge \neg(x_2 \oplus y_2) \wedge (x_1 \oplus y_1) \wedge x_1. \end{aligned} \quad (1)$$

Each line is a series of AND gates, so using the AND gate primitive enables us to compute the intermediate results. At last XORing all the bits yields the final comparison result.

In the AND gate protocol, $\mathcal{P}_2$ learns the number of expressions outputting 1 in the inequality circuit, but it is possible to make the pattern unidentifiable and hide it in the specific case of inequality comparison. Firstly, we pad all gates to the same length of l so that the '1' output by a certain AND gate does not reveal the gate itself. Secondly, we randomly reverse the inequality direction so that the AND gate computation becomes unidentifiable. If the result of the inequality (0 or 1) is random, the number of '1's output by AND gates is also random, so $\mathcal{P}_2$ cannot know the correct comparison result from the AND gate outputs, and the computation becomes oblivious with no leakage.

In the inequality computation, the initial step for $\mathcal{P}_0$ and $\mathcal{P}_1$ is to make a random decision for each pair of inputs x and y using their pre-negotiated seeds. If they generate a bit t that equals 1, they exchange the numbers, thus changing the direction of the inequality and the AND gate outputs. Then, they execute the protocol of AND gate to compute l series of AND gates for each pair of x and y , as (1). All gates are padded to length l . As $\mathcal{P}_2$ does not know the seed, it only obtains a masked result, denoted as $b' := b \oplus t$. $\mathcal{P}_2$ cannot distinguish the padded AND gates, and the computation becomes oblivious with no leakage. Finally, $\mathcal{P}_0$ and $\mathcal{P}_1$ can convert back to the correct direction after computing the AND gates as $b' \oplus t$. The communication costs $l(l+3)+2$ bits. We illustrate the protocol of inequality computation in Appendix A.

The approach of reversing the direction works well in most cases, except in the condition where two numbers equal to each other. In this case, they may be wrongly considered to satisfy the inequality. This issue happens either in the comparison of a query value and the database or in the MIN/MAX comparison of two stored values. For the former case, the data can be encoded in a spaced manner to avoid the error. Specifically, the data is encoded as even numbers, and the query number is encoded as

TABLE I
COST COMPARISON OF PETRA'S NEW PRIMITIVES WITH OTHER APPROACHES. l IS THE INPUT BIT-LENGTH, n IS THE NUMBER OF AND GATES. THE COST IS THEORETICALLY AMORTIZED TO EACH DATABASE ELEMENT.

		SECURITY	Serial	Petra
Equality	Round	$O(\log l)$	$O(l)$	2
	Comm.	$O(l \log l)$	$O(l)$	$2l + 2$
AND gate	Round	$O(\log n)$	$O(n)$	2
	Comm.	$O(n)$	$O(n)$	$2n + 2$
Inequality	Round	$O(\log l + 1)$	$O(l)$	2
	Comm.	$O(l \log l)$	$O(l)$	$l(l + 3) + 2$

Input: $\mathcal{P}_0$ and $\mathcal{P}_1$ hold sharings of two n -element secret vectors $\llbracket \vec{b} \rrbracket$ and $\llbracket \vec{z} \rrbracket$.
Output: $\mathcal{P}_0$ and $\mathcal{P}_1$ obtain a vector share $\llbracket \vec{z}' \rrbracket$ to the two parties, where for each component, $z'_i = z_i$ if $b_i = 1$ and 0 otherwise.

Fig. 6. Input and Output of Protocol Multiplexer.

an odd number. For the latter case, in fact this issue does not change the result of MIN/MAX operators.

4) *Cost Analysis:* Table I compares the costs of our primitives with other approaches of replicated secret sharing (RSS). Most RSS-based prior works [16], [36], such as Secrecy [16], implement the primitives using parallelized boolean circuits. Another implementation approach is using serial circuits. These approaches inherently require multiple rounds [37], [38]. By contrast, Petra's primitives only cost two communication rounds. The Equality and AND gate primitives require transmitting two pseudorandom numbers. In the Inequality primitive, the communication overhead is higher as the parties need to compute l boolean expressions, but the communicating messages for each party are still two. In Section VI, we will analyze the practical performance of our primitives and conduct a detailed comparison.

C. Privacy-Preserving Relational Operators

This section illustrates the privacy-preserving protocols for relational operators based on our new primitives.

1) *SELECT:* In Petra, a SELECT operation with a predicate p referring to certain attributes of R outputs a new relation R' along with a vector of bits. The output vector indicates for each row whether it is included in the output relation R' (i.e., valid). The vector is secret-shared, so its true value remains hidden (Fig. 6). The key step in the SELECT operation is computing a multiplexer [35] given an indicator bit b . The inputs to the multiplexer include secret shares $\llbracket \vec{b} \rrbracket$ and $\llbracket \vec{z} \rrbracket$ from $\mathcal{P}_0$ and $\mathcal{P}_1$. The multiplexer outputs two-party shares $\llbracket \vec{z}' \rrbracket$, where for each component, $z'_i = z_i$ if $b_i = 1$ and 0 otherwise.

We design a protocol for multiplexer. We firstly consider that the bits $\vec{b}$ are computed via equality comparisons or AND gates, such as SELECT . . . WHERE $R.a > 30$. Then we consider the

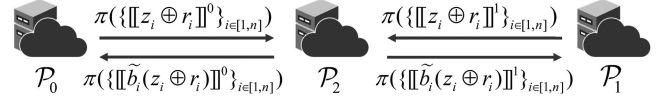


Fig. 7. An Illustration of Protocol Multiplexer.

case where each b_i is obtained via inequality comparisons with a random bit t_i , such as SELECT . . . WHERE $R.a > 30$.

Multiplexer Combined with Equality: The multiplexer operation, which selects data based on a condition bit b_i , consists of five steps. In the first step, $\mathcal{P}_0$ and $\mathcal{P}_1$ mask their shares of z_i as $\llbracket z_i \oplus r_i \rrbracket^0 = \llbracket z_i \rrbracket^0 \oplus r_i \oplus s_i$ and $\llbracket z_i \oplus r_i \rrbracket^1 = \llbracket z_i \rrbracket^1 \oplus s_i$, where s_i and r_i are pseudorandom numbers generated by their seeds.

In the second step, they shuffle their masked shares using the same permutation as $\vec{b}$ and send to $\mathcal{P}_2$ (Fig 7).

In the third step, $\mathcal{P}_2$ reconstructs the masked value $z_i \oplus r_i$ by XORing the received shares and performs the core multiplexer logic. It expands the plaintext selection bit b_i (obtained from the prior equality step or AND gate) to a word $\tilde{b}_i$ of the same bit-length as z_i by replicating b_i . It then computes the bitwise product $\tilde{b}_i(z_i \oplus r_i)$. We take a simple numerical example to describe this step. Let bit-length $l = 4$, $z_i = 1011_2$ and the masking value $r_i = 1100_2$. Then, $\mathcal{P}_2$ learns $z_i \oplus r_i = 0111_2$ and the original z_i remains hidden behind r_i . Depending on the value of b_i , $\mathcal{P}_2$ computes $\tilde{b}_i(z_i \oplus r_i)$ as follows:

- If $b_i = 0$, then $\tilde{b}_i = 0000_2$, and $\tilde{b}_i(z_i \oplus r_i) = 0000_2 \cdot 0111_2 = 0000_2$.
- If $b_i = 1$, then $\tilde{b}_i = 1111_2$, and $\tilde{b}_i(z_i \oplus r_i) = 1111_2 \cdot 0111_2 = 0111_2$.

In the fourth step, $\mathcal{P}_2$ splits the intermediate result $\tilde{b}_i(z_i \oplus r_i)$ into two shares and sends back to $\mathcal{P}_0$ and $\mathcal{P}_1$. In the fourth step, $\mathcal{P}_0$ and $\mathcal{P}_1$ compute $\llbracket \tilde{b}_i \rrbracket r_i$, and output the result share

$$\llbracket z'_i \rrbracket = \llbracket \tilde{b}_i z_i \rrbracket = \llbracket \tilde{b}_i(z_i \oplus r_i) \rrbracket \oplus \llbracket \tilde{b}_i \rrbracket r_i.$$

In the last step, following the proposed working pattern, $\mathcal{P}_0$ and $\mathcal{P}_1$ performs re-shuffling to ensure the output follows the right order as Fig 7 shows.

The input z_i can also be arithmetic shares, in which case the output becomes $\llbracket b_i \times z_i \rrbracket$. This variant of SELECT enables us to combine with arithmetic aggregation queries that operate on arithmetic shares, such as SUM and COUNT. More concretely, $\mathcal{P}_0$ and $\mathcal{P}_1$ mask each operand z_i as $z_i + r_i$. Then the multiplication-style multiplexer is to compute $\llbracket b_i \times z_i \rrbracket = \llbracket \tilde{b}_i \times (z_i + r_i) \rrbracket - \llbracket \tilde{b}_i \rrbracket \times r_i$.

Multiplexer Combined with Inequality: When the indicator bits are obtained from inequality computation, the multiplexer is more complicated. This is because we introduce an extra random bit t_i for each element to mask the direction of each inequality. When multiplying the masked result (denoted as b'_i) with the corresponding operand z_i , the bit t_i should be removed. We replicate the bits b'_i and t_i to two numbers $\tilde{b}'_i$ and $\tilde{t}_i$, and then the target multiplexer equals to $(\tilde{b}'_i \oplus \tilde{t}_i)z_i$. For secret sharings, the

parties need to compute:

$$\llbracket (b'_i \oplus \tilde{t}_i) z_i \rrbracket = \llbracket b'_i (z_i \oplus r_i) \rrbracket \oplus \llbracket \tilde{b}'_i r_i \oplus \tilde{t}_i \llbracket z_i \rrbracket \rrbracket.$$

This is similar to the case of computing multiplexer after equality comparison, except adding an extra term $\tilde{t}_i \llbracket z_i \rrbracket$.

The operand z_i can also be arithmetic shares, in which case the output becomes $(b'_i \oplus t_i) \times z_i = (b'_i + t_i - 2b'_i \times t_i) \times z_i$. The parties can first compute $\llbracket b'_i \times z_i \rrbracket$ using the multiplexer. Then $\mathcal{P}_0$ and $\mathcal{P}_1$ multiply the result with $(1 - 2t_i)$ and add another locally computed item $t_i \times \llbracket z_i \rrbracket$.

2) *Logical Composition*: Logical composition entails AND and OR operations that chain predicates together. OR can be converted into a combination of AND and NOT via De Morgan's law. NOT is a local negation of secret sharing (Section III), and the logical AND of two predicates can be viewed as a boolean AND gate. Consider this logical composition as an example:

$$(\mathbf{x} < 90 \text{ OR } \mathbf{y} < 40 \text{ OR } \mathbf{y} > 90)$$

$$\text{AND } (\text{time BETWEEN } 07-01 \text{ AND } 08-31).$$

Petra converts this composition to:

$$\text{NOT}(\mathbf{x} \geq 90 \text{ AND } \mathbf{y} \geq 40 \text{ AND } \mathbf{y} \leq 90)$$

$$\text{AND } (\text{time} \geq 07-01 \text{ AND } \text{time} \leq 08-31).$$

3) *MIN/MAX Aggregation and Sort*: MIN and MAX (used in aggregation and Sort) require an inequality comparison followed by a multiplexer, i.e., $\max = bx \oplus by$, where b is a bit indicating whether $x > y$ (i.e., $b = x > y ? 1 : 0$). For MIN/MAX aggregation, we perform the traversal by parallel computation in $\log |R|$ iterations [18].

The Sort operator of Petra sorts a relation R with multiple sorting attributes $(a_1, \dots)$ in ascending $\uparrow$ or descending $\downarrow$ order: $\text{Sort}_{\uparrow/\downarrow a_1 \dots}(R)$. To compute Sort, we need a swap operation, which can be realized by MIN and MAX. Given two operands x, y and their corresponding attributes a_x and a_y , we output $b_x = a_x, b_y = a_y$ if $x < y$, and swap them otherwise. Then we can realize Sort using a parallel swap-based sort algorithm called bitonic sort [16]. To conduct a bitonic sort, the protocol requires $O(\log n \cdot (\log n + 1))$ rounds of swaps, and the total complexity is $O(n \log n \cdot (\log n + 1))$ swaps, where $n = |R|$.

4) *Join*: The operator Join scans a cartesian product of two relations $R1, R2$, finds which rows should be joined according to predicate θ , and outputs a new column that indicates whether the row is joined and valid. It is formally defined as: $R1 \bowtie R2 = \{(r, r', \theta(r, r')) \mid r \in R1, r' \in R2\}$. In order to protect the parties from knowing which rows of each input match, the output is a cartesian product of the two joining inputs. The join operator is in essence a comparison between two inputs to find the matching rows. Since our primitives of Equality work well over large volumes of data, so we use a bulk of equality comparisons for Join. After computing the comparison, there are $|R2|$ indicator bits for each element of $R1$ and we can use them for further masking or logical composition. The cost is $|R1| \cdot |R2|$ equality comparisons.

A variant of join is Semi-Join. The (left) semi-join scans a cartesian product of two input relations $R1, R2$, and only returns rows of $R1$, some of which might successfully join with $R2$ or not. The output is a relation with the same cardinality as

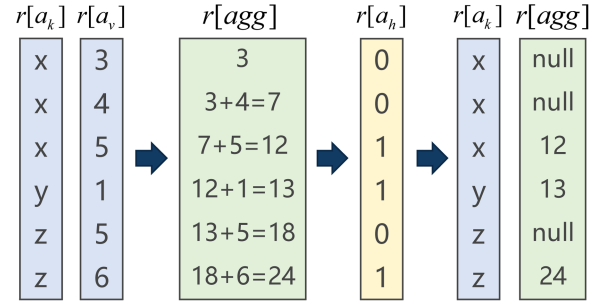


Fig. 8. An Example of Group-by and Sum.

$R1: R3 = \{r, \bigvee_{r' \in R2} \theta(r, r') \mid r \in R1\}$, where the new attribute $\bigvee \theta(r, r')$ denotes whether the tuple in $R1$ matches any tuple in $R2$. Semi-Join still requires $|R1| \cdot |R2|$ comparisons like Join, plus a logical composition to compute the new indicating attribute. Computing the conjunction $\bigvee \theta(r, r') = \neg(\neg b_1 \wedge \neg b_2 \wedge \dots \wedge \neg b_{|R2|})$ can use Petra's AND gate primitive only needs two rounds. The total cost is $|R1| \cdot |R2|$ equality comparisons and AND gates for $|R1|$ elements.

5) *Group-by*: In most cases, Group-by is combined with an aggregation function $f(\cdot)$ on the rows of each group, and we denote the operator as $\gamma_{a_k, a_v, f}(R)$. Let the group-by attribute in the input relation R be a_k and the aggregated attribute be a_v . Assuming that there are m unique values in the column of a_k , $\gamma_{a_k, f}(R)$ splits the relation to m subgroups such that in each group the elements have identical values for a_k , and then performs aggregation within each group. The output relation is equipped with two new attributes: the indicator a_h and the aggregation result agg . Among the output, there are m valid tuples with the value $r[a_h]$ equal to one if it is the last member of its group, and its value $r[agg]$ equal to the group's aggregation result.

We now explain the protocol of Group-by. Fig. 8 illustrates an example of grouping by category and computing sums. Assume the keys are in order before uploading to $\mathcal{P}_0$ and $\mathcal{P}_1$. If the keys are not in order, the first step is to perform sorting on the keys. (For aggregation functions like MIN/MAX, the values of a_v also need to be sorted so that, within each group sharing the same key, the values are in order.) After sorting, there are three more steps. In the first step, $\mathcal{P}_0$ and $\mathcal{P}_1$ traverse the data and build a new attribute agg : for SUM, they add values one by one; for MAX, they set the value of agg to the value of a_v . (MIN can be realized in an analogous way.) Although at this time agg is not yet the final output, it provides an intermediate aggregation result. In the second step, the three parties perform $|R| - 1$ batch equality checks using the Equality primitive. This primitive compares each tuple's key $r_i[a_k]$ with that of the following tuple $r_{i+1}[a_k]$, yielding an attribute a_h . If $r_i[a_k]$ does not equal to $r_{i+1}[a_k]$, $r_i[a_h]$ is set as 1, meaning r_i is the last of its group and the intermediate aggregation result agg can be split from $r_i[agg]$. In the third step, they perform a multiplexer according to a_h , keeping $r_i[agg]$ if $r_i[a_h] = 1$ and mask other tuples' values so that only groupwise aggregation results are remained. For SUM, the requester performs a post-processing:

computing a subtraction of agg to obtain the final results. For example, $r_4[agg] - r_3[agg] = 13 - 12 = 1$, which means the sum of a_v in the group with key $r_4[a_k] = y$ equals to one.

The cost of Group-by involves $O(n)$ operations of equality comparisons and multiplexer, where n is the number of tuples in the relation $|R|$.

D. Composition of Operators

The relational operators can be composed according to the query execution flow. Next we describe our optimizations of composing Petra's operators.

1) *Putting and Composition Together*: Composing multiple operators is in essence a logical composition of the indicator bits, which can be realized by our logical composition block. Typically, the composition is through a logical AND. We propose to gather the AND composition together as much as possible so that we could use our AND gate primitive.

When composing SELECT and Join by explicit AND gates, the SELECT filters can be packed together into the comparison of Join. That is, the comparison expression is packed as $(R1.r_i[a] = R2.r_j[a]) \wedge p1 \wedge p2 \dots$, where p is the output bit of filter predicates. If p is an equality predicate, it can be further fused by putting the equality expression inside.

When composing Group-by and other operators (such as SELECT and Join), we pack the composition within each equality comparisons. Concretely, when computing equality checks between $r_i[a_k]$ and $r_{i+1}[a_k]$, we take the previous operators into account by a logical composition. If the composition is a series of AND, it can be fused by our schemes of logical composition. Consider the following complex Query 1 as an example.

```
SELECT R1.k, R1.a, R1.sid, COUNT(*)
FROM R1 left join R2 ON R1.sid=R2.sid
AND R1.a>p AND R1.a<q
GROUP BY R1.k.
```

In the first phase, we compose the join and the filtering predicate to obtain the valid bit $r_i[b] \leftarrow r_i[join] \wedge r_i[p] \wedge r_i[q]$. $join$ is the result of the left-join operator, while $r[p]$ and $r[q]$ are the results of the two inequality predicates $R1.a > p$ and $R1.a < q$, respectively. In the second phase, we sort on the indicator bit and the keys to ensure valid items are placed prior to other items. Then we compose the indicator bit with the Group-by operator. As we perform equality comparison of each pair r_i, r_{i+1} , we combine other operators together: $(r_i[k] = r_{i+1}[k]) \wedge r_i[b] \wedge r_{i+1}[b]$. This can be performed within the equality comparison and does not introduce extra communication rounds for the Group-by operator.

2) *Fusing by Using XOR in Logical Composition*: As mentioned in Section IV-C2, the secure computation of OR gate is transformed into a combination of AND gate and local NOT gate. However, there are instances where XOR gate can be employed as a substitute for the OR gate and helps to save many AND gate operations. One is logical composition of predicates of exclusive conditions, such as $R1.a = q1 \text{ OR } R1.a = q2 \dots \text{ OR } R1.a = q8$. The value of attribute $R1.a$ cannot equal to multiple query values, so the OR gates can be

$\mathcal{F}_{ineq}$ receives secret vectors $[\tilde{x}]^0, [\tilde{y}]^0$ and $[\tilde{x}]^1, [\tilde{y}]^1$ of length n , from $\mathcal{P}_0$ and $\mathcal{P}_1$ respectively, reconstructs $\tilde{x}$ and $\tilde{y}$, and performs a component-wise comparison to produce a binary vector $\tilde{b}$. If $x_i > y_i$, b_i equals to 1. Otherwise, $[b_i]$ equals to 0. The functionality splits b into two vector shares $[\tilde{b}]^0$ and $[\tilde{b}]^1$, and sends to $\mathcal{P}_0$ and $\mathcal{P}_1$.

Fig. 10. Functionality of the Inequality Primitive ($\mathcal{F}_{ineq}$).



Fig. 11. An Illustration of the Leakage.

Firstly, $\mathcal{F}_{eq}$ receives n -element secret vectors $[\tilde{x}]^0, [\tilde{y}]^0$ and $[\tilde{x}]^1, [\tilde{y}]^1$, from $\mathcal{P}_0$ and $\mathcal{P}_1$ respectively, reconstructs $\tilde{x}$ and $\tilde{y}$, and performs a component-wise comparison to produce a binary vector $\tilde{b}$. If $x_i = y_i$, b_i equals to 1. Otherwise, $[b_i]$ equals to 0. The functionality splits b into two vector shares $[\tilde{b}]^0$ and $[\tilde{b}]^1$, and sends to $\mathcal{P}_0$ and $\mathcal{P}_1$. Secondly, $\mathcal{F}_{eq}$ also computes the leakage function $\text{Leak}(\mathcal{F}_{eq}) := \sum_{i=0}^n b_i$ and sends to $\mathcal{P}_2$.

Fig. 9. Functionality of the Equality Primitive ($\mathcal{F}_{eq}$).

replaced by XOR gates. Compared with the naive method of computing the expression by De Morgan's Law, this approach saves 7 OR gates. Another instance, as a special case of the aforementioned one, is joining on unique keys. Consider Query 2 as an example:

```
SELECT... FROM R1, R2 WHERE R1.a=R2.a.
```

If the keys of R2 are unique (or have been deduplicated by previous operations such as Group-by), the output size could be reduced to $|R1|$, and for each element of the output we perform a logical composition of equality comparisons related to elements of R2. The composition can be finished by local XOR gates since the equal condition is exclusive with each other. Doing this avoids the expensive composition by $|R2|$ AND gates.

V. SECURITY ANALYSIS

Our stand-alone security model follows the paradigm of real/ideal-world simulation [39]. The adversary $\mathcal{A}$ can possibly corrupt the members of the adversary structure Δ . In protocol P , it can simulate an ideal view Ideal using a simulator algorithm Sim that interacts with the ideal functionality $\mathcal{F}_P$. Correspondingly, the view in the real protocol is denoted as $\text{Real}(P, \mathcal{A})$. Formally, the security of protocol P requires that for a computationally bounded adversary $\mathcal{A}$, there exists a simulator algorithm $\mathcal{S}$ such that the two views are indistinguishable: $\text{Real}(P, \mathcal{A}) \approx \text{Ideal}(\mathcal{F}_P, \text{Sim}, \mathcal{A})$. Next, we define the ideal functionalities of Petra's primitives and describe their semi-honest security using simulation-based proofs.

Theorem 1: The protocol of equality comparison securely realizes the functionality $\mathcal{F}_{eq}$ (Fig. 9) in the honest-but-curious

TABLE II
PERFORMANCE OF DATABASE PRIMITIVES

		SECURITY [16]	SECURITY (optimized.) [36]	Serial	Petra
Equality					
2^{10} (1K)	LAN	0.0025s	0.0017s	0.004s	0.0004s
	WAN	0.12s	0.12s	1.3s	0.04s
	Comm.	0.14MB	0.14MB	0.024MB	0.016MB
2^{20} (1M)	LAN	1.55s	0.36s	0.14s	0.096s
	WAN	2.6s	1.2s	4.2s	0.3s
	Comm.	144MB	144MB	24MB	16MB
Inequality					
2^{10} (1K)	LAN	0.0029s	0.0026s	0.004s	0.002s
	WAN	0.15s	0.15s	1.3s	0.1s
	Comm.	0.14MB	0.14MB	0.048MB	1MB
2^{20} (1M)	LAN	1.6s	0.5s	0.16s	1.4s
	WAN	3.2s	1.5s	4.2s	2.4s
	Comm.	168MB	168MB	48MB	1072MB

model against the adversary structure $\Delta: \{\mathcal{P}_0\}, \{\mathcal{P}_1\}, \{\mathcal{P}_2\}\}$, with the leakage function $\text{Leak}(\mathcal{F}_{eq})$.

Proof: Firstly, we construct a simulator for the permissible corrupting parties $\{\mathcal{P}_0\}$ and $\{\mathcal{P}_1\}$. The real view of the corrupted party consists of secret shares $\{\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket b \rrbracket\}$ and locally computed pseudorandom numbers. The backend of two-party secret sharing guarantees the existence of a simulator Sim that can emulate an ideal view when provided with the corrupting party's inputs by interacting with the ideal functionality $\mathcal{F}_{eq}$.

Secondly, we analyze the security against the corrupted $\mathcal{P}_2$. It interacts with the ideal functionality's leakage function and learns a count, i.e., the number of elements that adhere to the equality relationship. In addition, the realistic view of $\mathcal{P}_2$ includes two shuffled lists of pseudorandom numbers: $\pi(S^0), \pi(S^1)$. As a result, we construct the simulator Sim as follows. Let the count from the functionality be m , Sim samples n pairs of random numbers from the same field of $\llbracket x \rrbracket, \llbracket y \rrbracket$, among which m pairs are composed of two identical numbers and the rest are composed of two different numbers. It sends these pairs as a simulated view to $\mathcal{P}_2$, who then performs a component-wise comparison and generates a binary vector $\vec{b}$ according to the order of the PRF results. By the property of pseudorandomness, the simulated view and the real view are indistinguishable to each other. $\square$

The security of AND gate protocol is similar. The leakage of the two primitive reveals a volume pattern to $\mathcal{P}_2$ during computation, which refers to the number of intermediate results. For example, for the predicate $x=100 \text{ AND } y=30$, $\mathcal{P}_2$ knows how many records return true because they satisfy the condition.

Theorem 2: The protocol of inequality comparison securely realizes $\mathcal{F}_{ineq}$ (Fig. 10) in the honest-but-curious model against the adversary structure $\Delta: \{\mathcal{P}_0\}, \{\mathcal{P}_1\}, \{\mathcal{P}_2\}\}$.

Proof: The condition for the adversary corrupting parties $\{\mathcal{P}_0\}, \{\mathcal{P}_1\}$ is similar to that in Theorem 1, and the simulator can be constructed in a similar way.

The security against the adversary corrupting $\mathcal{P}_2$ is more complicated. Since each inequality boolean circuit consists of l series of AND gates, $\mathcal{P}_2$'s real view includes two lists of

$n \times l$ series of l -bit pseudorandom numbers. Due to padding and the reversing bit mask, the view is uniformly random. We construct the simulator Sim as follows. It randomly samples as many pairs of random λ -bit numbers as $l|R|$, among which the number of pairs with two identical numbers is also random (R is the database size). It sends these random numbers to the adversary. Then $\mathcal{A}$ generates an indicator vector according to the order of the PRF results and outputs the vector shares. By the property of pseudorandomness, it follows that the simulated view is indistinguishable from the real view obtained by $\mathcal{A}$ in the protocol. $\square$

Now we describe the security of Petra's operators. The Multiplexer protocol has an extra operand z combined with equality or inequality comparisons. The operand z is secretly shared between $\mathcal{P}_0$ and $\mathcal{P}_1$, and also masked before sending to $\mathcal{P}_2$. Therefore, it is secure in our security model. (We provide a theorem and a proof in the appendix, available online.) The join operator performs $|R1| \times |R2|$ times equality checks, revealing to $\mathcal{P}_2$ learns the number of the matching tuples in the cartesian product of two relations. The group-by operator performs $|R| - 1$ equality checks to identify the groups. Then $\mathcal{P}_2$ learns the number of the groups, i.e., the distinct element count.

Petra executes a complex query by decomposing it into a sequence of relational operators. We can complete the overall system security via a composition argument of these operators. Information leakage may occur in operators that reduce the data volume, such as SELECT, Join, and Group-by, as well as in their logical compositions. This is because they leverage our equality primitive and the AND gate primitive. In general, Petra is secure against the adversary structure's security model, subject to leakage to $\mathcal{P}_2$: the union of volume information from each instance of the Equality protocol and the AND gate protocol. The detailed proofs and analysis are provided in Appendix B. Crucially, this composition guarantee extends to arbitrarily complex operators because each operator instance in Petra is executed with fresh randomness and independent PRF keys, and intermediate results are re-shuffled before being passed to subsequent operators. This ensures that $\mathcal{P}_2$'s view across multiple stages remains limited to the union of per-operator cardinalities. Specifically, the leakage from the underlying primitives presents $\mathcal{P}_2$ with the resulting data volume of filtering or joining steps, while $\mathcal{P}_0$ and $\mathcal{P}_1$ learn nothing. Fig. 11 illustrates an example of this leakage. When applying the first operator, which is secure, there is no information leakage. When applying the second operator and composing it with the third operator, the computation leaks the number of valid data items to $\mathcal{P}_2$ in these two steps. The leakage only indicates the size of the result set after each operator without revealing specific data, and makes sense to achieve highly efficient query execution.

Discussion on Leakage Mitigation: To effectively control the leakage, one solution is introducing dummy data items to perturb the volume information that $\mathcal{P}_2$ learns [12], [40], [41]. The key idea is to have Party 0 and Party 1, in each instance of the Equality and AND gate primitives, insert dummy items before computing the pseudorandom function. Some of the dummy items are programmed to yield a "match" result in the comparison, increasing

the volume of valid data, while others yield a “non-match” result. The approach of generating the dummy items is determined by differential privacy mechanisms [42], introducing a noise to the volume information. The mechanism calibrates the differential noise based on the privacy budget and the sensitivity of the operator. For queries involving multiple operators, the noise can be composed using composition theorems. The formal design of this mechanism and a rigorous privacy analysis remain open questions for future work. Additionally, a deployment variant could instantiate $\mathcal{P}_2$ inside a trusted execution environment (TEE). This selective hardening strengthens the subsystem that receives the defined leakage, while crucially avoiding the single point of trust and excessive data exposure that would result from placing the entire computation within a single TEE.

VI. PERFORMANCE EVALUATION

A. Implementation

We implement a prototype of Petra in C++, realizing Petra’s privacy-preserving primitives and relational operators. We also conduct experiments with many relational queries over randomly initialized data. All secret sharings in our implementation are defined over the field 2^{64} , and all relational tables are initialized randomly.

Our evaluation is conducted on three virtual machines hosted on a physical server with an Intel Xeon 6530 2.60 GHz CPU. Each machine, representing a computing party, has 8 GB of RAM. We simulate two network settings for cloud deployments. One is a LAN with a maximum bandwidth of 10 Gbps and 0.5 ms round-trip latency. The other is a WAN with a maximum bandwidth of 5 Gbps and 40 ms round-trip time [20], [22].

We compare our proposed schemes against the state-of-the-art approach based on Replicated Secret Sharing (RSS). RSS is widely adopted for its high efficiency and effectiveness in secure multi-party computation. It operates in a three-party system model, which is the same as ours, making it a relevant baseline for our evaluation. SECRECY [16], [43] leverages RSS under the same honest-majority setting and provides a practical relational system. We use SECRECY for a direct and meaningful comparison of individual operators and end-to-end query execution. The communication overhead involves the summation of all channels’ communication between the computing parties. It is important to note the difference in privacy guarantees: SECRECY provides no leakage, while Petra operates with an auxiliary party $\mathcal{P}_2$ that receives limited, predefined leakage. The comparison thus highlights the trade-off between performance and privacy, with Petra offering a practical solution for scenarios where the defined leakage is acceptable.

B. Performance of Database Primitives

We evaluate Petra’s primitives from Section IV-B and compare them with RSS-based methods. Secrecy [16] realizes the primitives using parallel boolean circuits and the recent work ORQ [36] further optimizes the implementation. Another implementation of the primitives is using serial circuits. The secret sharing is configured as 64-bit, and the size of the input relation

table is 2^{17} (130K) or 2^{20} (1 M). The measurements are shown in Table II.

1) *Equality Primitive*: As analyzed in Section IV-B4, Petra’s equality primitive requires only two communication rounds regardless of input size, with total communication $O(n\ell)$. This contrasts with RSS-based approaches [16], [36] that require $O(\log \ell)$ rounds (parallel circuits) or $O(\ell)$ rounds (serial circuits). The experimental results validate this advantage: for 2^{20} elements in LAN, Petra completes in 96 ms, outperforming Secrecy’s parallel implementation (1.55 s) and the optimized implementation (0.36 s). The advantage is even more pronounced in WAN settings where latency dominates. Petra (0.3 s) is $4\times$ faster than [36] (1.2 s) for 1 M inputs.

2) *Inequality Primitive*: The inequality primitive presents a more nuanced trade-off. As analyzed in Section IV-B4, the RSS-based computation using serial circuits minimizes total communication, while Petra minimizes round complexity. The optimal choice depends on the deployment environment and input scale. For 2^{10} elements, Petra (0.002 s in LAN, 0.1 s in WAN) outperforms both Secrecy’s parallel implementation [16] and the improved variant [36], as the overhead of additional rounds outweighs Petra’s higher per-round communication for small n . As n increases to 2^{20} , the landscape shifts. In LAN, Petra (1.36 s) falls behind the optimized implementation (0.5 s) and the serial circuit approach (0.16 s). This is expected: in low-latency environments with large n , total communication volume becomes the dominant factor. Petra’s $O(n\ell^2)$ communication exceeds that of these approaches, and this bandwidth overhead outweighs the round complexity advantage. In WAN, Petra (2.4 s) outperforms Secrecy [16] (3.2 s) and remains competitive with the optimized implementation (1.5 s). The high round-trip latency amplifies the cost of additional rounds, allowing Petra’s constant-round design to partially offset its bandwidth disadvantage. As n grows further to ten millions (10 M), the communication overhead still becomes increasingly prohibitive for Petra. It requires 28 seconds, while the optimized implementation takes 22 seconds. In summary, Petra’s inequality primitive excels in low-latency environments and smaller input sizes, while it may not be the optimal choice for large n in high-bandwidth, low-latency settings. The choice of primitive can be determined by the specific deployment context and performance requirements.

C. Performance of Operators

1) *SELECT and Aggregation*: In this section, we run aggregation queries and compare the performance, using SUM and MIN/MAX aggregation, some of which has composite filters of equality or inequality predicates. Each filter consists of two predicates. Fig. 12 shows the runtime for an input size of 1 million records, and Table III evaluates the communication overhead.

The execution time for SUM aggregation with predicates primarily involves SELECT with comparisons. Petra has better performance due to its multiplexer-based SELECT integrated with our comparison primitives. In LAN (Fig. 12(a)), Petra computes a query with equality predicates on 1 M records

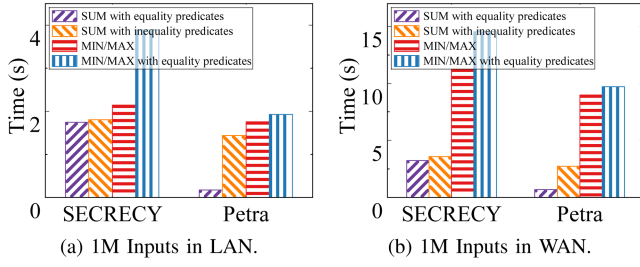


Fig. 12. Performance of Aggregation (1 M inputs).

TABLE III
PERFORMANCE OF MIN/MAX AGGREGATION

Input	Network	SECRECY [16]			Petra		
		Time	Rounds	Comm.	Time	Rounds	Comm.
2^{17}	LAN	0.32s	136	24MB	0.27s	34	132MB
	WAN	3.73s			1.76s		
2^{20}	LAN	2.14s	160	192MB	1.14s	40	1056MB
	WAN	11.25s			9.00s		

TABLE IV
COMMUNICATION OF OPERATORS

	Input	SECRECY [16]		Petra	
		Rounds	Comm.	Rounds	Comm.
Sort	2^{16}	1088	918MB	272	4624MB
	2^{20}	1680	22680MB	420	114240MB
Join	$2^{13}, 2^{13}$	6	9216MB	2	1040MB
	$2^{16}, 2^{16}$	6	589824MB	2	66560MB
Semi-Join	$2^{13}, 2^{13}$	19	9240MB	4	2080MB
	$2^{16}, 2^{16}$	22	591360MB	4	132098MB
Group-by	2^{16}	96	157MB	2	3MB
	2^{20}	120	3192MB	2	48MB

in 0.1 s, compared to 1.7 s for SECRECY takes, achieving 10 times faster. For queries with inequality predicates, Petra takes 1.4 s, while SECRECY takes 1.8 s. In WAN (Fig. 12(b)), Petra computes an aggregation query on 1 M records in 0.7 s (equality predicates) and 2.7 s (inequality predicates).

MIN/MAX aggregation generally requires more time and communication overhead. Petra's performance exceeds SECRECY on these queries. For 130 K records, Petra requires 34 rounds, while SECRECY requires less communication but 4 times more rounds (Table III). This helps Petra achieve an improvement in overall query time. Specifically, Petra computes the query in 0.27 s in LAN and 1.76 s in WAN (Table III), up to 2× faster than SECRECY. For 1 M records, Petra takes 1.14 s in LAN (Fig. 12(a)) and 9 s in WAN (Fig. 12(b)). When filters are added, Petra's SELECT further reduces the time. Petra takes 1.9 s on 1 M records in LAN and 9.7 s in WAN, while SECRECY takes 3.8 s in LAN and 14.5 s in WAN.

2) *Other Operators*: In this section, we run four protocols of Petra's operators (Sort, Join, Semi-Join, Group-by) with increasing input sizes in two network environments and report the total execution time in Fig. 13. We also compare the performance with the protocols of SECRECY. Table IV analyzes the communication costs.

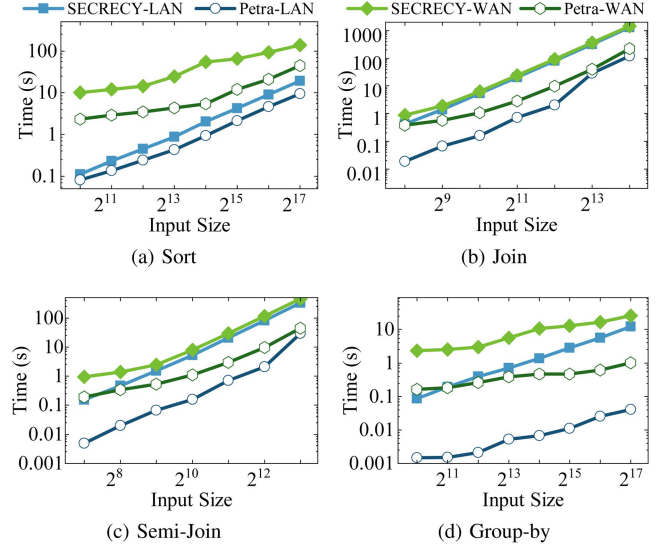


Fig. 13. Performance of Operators.

We firstly evaluate the performance of Sort. Petra's Sort requires fewer communication interactions per stage of comparison and swap, resulting in a fourfold reduction in total communication rounds. As shown in Fig. 13(a), Petra's Sort achieves up to 9× better performance. As the input size increases to one million (2^{20}), Petra takes 150 s in LAN and 6 minutes in WAN, while SECRECY takes 160 s in LAN and 16 minutes in WAN. The general Join and Semi-Join operators, are relatively expensive, involving bulk comparisons of the Cartesian product of two relations. The communication rounds for Join are equivalent to those for comparison, while semi-join requires additional communication to compose $|R_2|$ comparison results. In this measurement, we set the two input relations to be of equal size. Fig. 13(b) and (c) show that the two operators have similar performance. Petra's Join is 11-40× faster in LAN and 3-9× faster in WAN compared to SECRECY. Our Semi-join is also 8×-10× faster, eliminating the need for $\log |R_2|$ rounds of composition and reducing communication overhead. Another observation is that SECRECY's execution time is similar across two networks, mainly because round-trip latency does not dominate the overall time. Petra's Group-by also outperforms SECRECY under both network conditions. The performance is illustrated in Fig. 13(d). Petra is over 80× faster in LAN and 20× faster in WAN.

D. Optimizations of Operator Composition

This section evaluates the communication costs of our two composition optimizations. Table V shows the evaluation results. We focus solely on the cost of composition, without considering the preceding operator. We test the first query example mentioned in Section IV-D. We compose Group-by with other operators by performing AND composition within each pairwise comparison, requiring no additional communication round. In contrast, SECRECY computes four AND gates for each tuple in R_1 , and for each comparison, it computes two more AND gates using two extra rounds of communication. As SECRECY's

TABLE V
COMMUNICATION OF OPERATOR COMPOSITION

	Input	SECRECY [16]		Petra	
		Rounds	Comm.	Rounds	Comm.
Query 1	2^{16}	35	135MB	0	70MB
	2^{20}	43	2751MB	0	1710MB
Query 2	$2^{13}, 2^{13}$	13	24MB	0	local computation
	$2^{16}, 2^{16}$	16	1536MB	0	local computation

Group-by involves multiple iterations of pairwise comparisons, this results in twice as many communication rounds. In our approach, these extra rounds are eliminated, saving $3 + 2 \log |R1|$ communication rounds.

The second test evaluates the impact of using XOR, specifically with a query involving a Semi-join on unique keys (Section IV-D). The original method computes $|R2|$ OR gates, incurring a cost of $\log |R2|$ rounds in SECRECY's RSS-based schemes. When using XOR, the AND composition is eliminated and the process requires only local computation. Therefore, the composition becomes effectively free in terms of communication cost.

E. Performance on Real Queries

In this section, we evaluate and compare the performance of Petra on three real-world queries from the public benchmark TPC-H [44]: Q4, Q6, Q13, which have also been used in SECRECY. For Q4 and Q13, which conduct computations over multiple relations, we keep the size of the smaller input fixed at 1024 while increasing the size of the other relation. The query execution pipeline is identical to that of SECRECY, covering five blocks: comparison predicates, group-by, logical composition, multiplexer and aggregation. The primary distinction between Petra and SECRECY lies in Petra's new protocols for operators and composition, which achieve higher communication efficiency. The results are depicted in Fig. 14.

Q4 is a query with a left semi-join transformed from a sub-query, aggregation and group-by, aggregation. When the larger database inputs are 2 K records, Q4 involves 2 M equality comparisons, logical compositions of 2 K bits for 1 K records and aggregation on 1 K records. In this case, Petra takes 0.5 s in LAN (Fig. 14(a)) and 5 s in WAN (Fig. 14(b)), achieving 6-20 $\times$ speedup than SECRECY. As the scale factor increases and the database size scales to 2^{17} , Petra completes in 23 s in LAN and 1.5 min in WAN. Q6 consists of SUM and SELECT with five filters of inequality predicates. For Q6, Petra outperforms SECRECY by 1.4 $\times$ in LAN (Fig. 14(c)) and 2.3 $\times$ in WAN (Fig. 14(d)). The query performance is similar to the aggregation queries with filters (Section VI-C1). Q13, as the most expensive query, involves applying group-by after left semi-join and performing aggregation with another group-by. Petra has more efficient operators than SECRECY and also eliminates the need for composition of bits in the outer semi-join. Fig. 14(e) shows that in LAN Petra completes the query on 130 K records in 40 s, reducing the time by 12 $\times$ compared to SECRECY. Fig. 14(f) shows that Petra requires 7 $\times$ less running time in WAN.

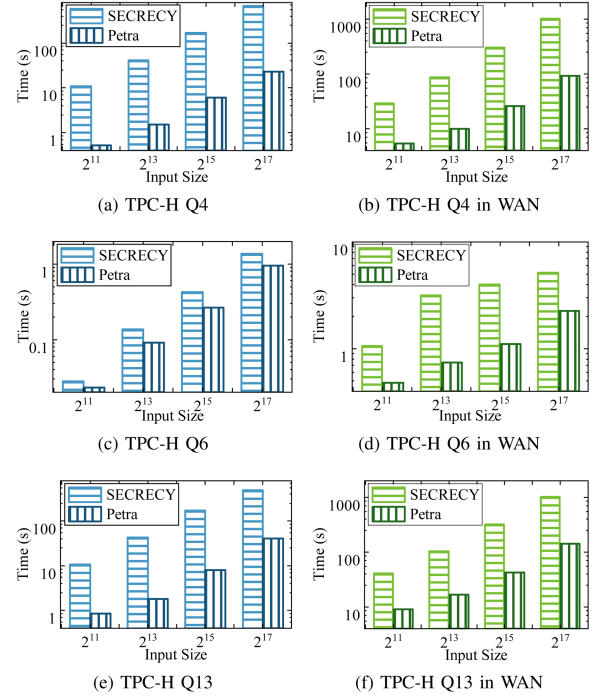


Fig. 14. Performance on TPC-H Query Benchmarks.

VII. CONCLUSION

We presented Petra, a new and efficient three-party relational analytics system with rich functionality for outsourced databases. We proposed new privacy-preserving primitives for basic operations on databases. The new primitives reduce interactions, thus largely improving overall efficiency. By designing multiplexer, aggregation, sort, and other operators specifically for relational analytics in Petra, we enabled SQL operators to be efficiently executed on relational databases. Although there is potential volume leakage to one computing party, we stated that this helps to improve the overall efficiency in practice, and we also proposed two remedies to mitigate or eliminate the leakage. Future directions include optimizing core operators with more efficient algorithms, integrating formal differential privacy guarantees, and extending Petra to support a wider range of SQL queries and analytical workloads. Experiments and empirical results showed that Petra achieves speedups of more than one order of magnitude compared to prior works for standard query benchmarks while maintaining privacy guarantees.

REFERENCES

- [1] S. Chaudhuri, "An overview of query optimization in relational systems," in *Proc. 17th ACM SIGACT-SIGMOD-SIGART Symp. Princ. Database Syst.*, ACM, 1998, pp. 34–43.
- [2] E. Meijer, B. Beckman, and G. Bierman, "LINQ: Reconciling object, relations and XML in the .NET framework," in *Proc. ACM Int. Conf. Manage. Data*, 2006, pp. 706–706.
- [3] F. Abuzaid et al., "DIFF: A relational interface for large-scale data explanation," *Proc. Very Large Data Bases Endowment*, vol. 12, no. 4, pp. 419–432, 2018.
- [4] D. Bogdanov, L. Kamm, B. Kubo, R. Rebane, V. Sokk, and R. Talviste, "Students and taxes: A privacy-preserving study using secure computation," in *Proc. Privacy Enhancing Technol. (PoPETs)*, 2016, pp. 117–135.

- [5] D. Bogdanov, L. Kamm, S. Laur, and V. Sokk, "Rmind: A tool for cryptographically secure statistical analysis," *IEEE Trans. Dependable Secur. Comput.*, vol. 15, no. 3, pp. 481–495, May–Jun. 2018.
- [6] G. Ling et al., "P²FRPSI: Privacy-preserving feature retrieved private set intersection," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2201–2216, 2024.
- [7] D. Demmler, T. Schneider, and M. Zohner, "ABY-A framework for efficient mixed-protocol secure two-party computation," in *Proc. Netw. Distrib. System Secur. Symp.*, 2015, pp. 1–15.
- [8] J. Doerner and A. Shelat, "Scaling ORAM for secure computation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, ACM, 2017, pp. 523–535.
- [9] P. Mohassel and P. Rindal, "ABY³: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, ACM, 2018, pp. 35–52.
- [10] M. Keller, "MP-SPDZ: A versatile framework for multi-party computation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, ACM, 2020, pp. 1575–1590.
- [11] J. Bater, G. Elliott, C. Eggen, S. Goel, A. Kho, and J. Rogers, "SMCQL: Secure querying for federated databases," *Proc. Very Large Data Bases Endowment*, vol. 10, no. 6, pp. 673–684, 2017.
- [12] J. Bater, X. He, W. Ehrlich, A. Machanavajjhala, and J. Rogers, "Shrinkwrap: Efficient SQL query processing in differentially private data federations," *Proc. Very Large Data Bases Endowment*, vol. 12, no. 3, pp. 307–320, 2018.
- [13] N. Volgushev, M. Schwarzkopf, B. Getchell, M. Varia, A. Lapets, and A. Bestavros, "Conclave: Secure multi-party computation on Big Data," in *Proc. 14th Eur. Conf. Comput. Syst.*, ACM, 2019, pp. 1–18.
- [14] J. Bater, Y. Park, X. He, X. Wang, and J. Rogers, "SAQE: Practical privacy-preserving approximate query processing for data federations," *Proc. Very Large Data Bases Endowment*, vol. 13, no. 12, pp. 2691–2705, 2020.
- [15] R. Poddar, S. Kalra, A. Yanai, R. Deng, R. A. Popa, and J. M. Hellerstein, "Senate: A maliciously-secure MPC platform for collaborative analytics," in *Proc. 30th USENIX Secur. Symp. (USENIX Security)*, 2021, pp. 2129–2146.
- [16] J. Liagouris, V. Kalavri, M. Faisal, and M. Varia, "SECRECY: Secure collaborative analytics in untrusted clouds," in *Proc. 20th USENIX Symp. Networked Syst. Des. Implementation (NSDI)*, 2023, pp. 1031–1056.
- [17] P. Mohassel, P. Rindal, and M. Rosulek, "Fast database joins and PSI for secret shared data," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, ACM, 2020, pp. 1271–1287.
- [18] F. Han, L. Zhang, H. Feng, W. Liu, and X. Li, "Scape: Scalable collaborative analytics system on private database with malicious security," in *Proc. 38th IEEE Int. Conf. Data Eng.*, 2022, pp. 1740–1753.
- [19] G. Asharov, K. Hamada, R. Kikuchi, A. Nof, B. Pinkas, and J. Tomida, "Secure statistical analysis on multiple datasets: Join and group-by," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 3298–3312.
- [20] E. Dauterman, M. Rathee, R. A. Popa, and I. Stoica, "Waldo: A private time-series database from function secret sharing," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 2450–2468.
- [21] X. Wang, A. J. Malozemoff, and J. Katz, "EMP-toolkit: Efficient multiparty computation toolkit." Accessed: Apr. 2026. [Online]. Available: <https://github.com/emp-toolkit>
- [22] M. Faisal, J. Zhang, J. Liagouris, V. Kalavri, and M. Varia, "TVA: A multi-party computation system for secure and expressive time series analytics," in *Proc. 32th USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 5395–5412.
- [23] Y. Wang and K. Yi, "Secure Yannakakis: Join-aggregate queries over private data," in *Proc. ACM Int. Conf. Manage. Data*, 2021, pp. 1969–1981.
- [24] Y. Li, D. Ghosh, P. Gupta, S. Mehrotra, N. Panwar, and S. Sharma, "PRISM: Private verifiable set computation over multi-owner outsourced databases," in *Proc. ACM Int. Conf. Manage. Data*, 2021, pp. 1116–1128.
- [25] H. Corrigan-Gibbs and D. Boneh, "Prio: Private, robust, and scalable computation of aggregate statistics," in *Proc. 14th USENIX Symp. Networked Syst. Des. Implementation (NSDI)*, 2017, pp. 259–282.
- [26] G. Asharov et al., "Efficient secure three-party sorting with applications to data analysis and heavy hitters," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 125–138.
- [27] B. Fuller et al., "SoK: Cryptographically protected database search," in *Proc. IEEE Symp. Secur. Privacy (SP)*, IEEE, 2017, pp. 172–191.
- [28] I. Demertzis, D. Papadopoulos, C. Papamanthou, and S. Shintre, "SEAL: Attack mitigation for encrypted databases via adjustable leakage," in *Proc. 29th USENIX Secur. Symp. (USENIX Security)*, 2020, pp. 2433–2450.
- [29] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, "CryptDB: Protecting confidentiality with encrypted query processing," in *Proc. ACM Symp. Operating Syst. Princ.*, 2011, pp. 85–100.
- [30] S. Kamara and T. Moataz, "SQL on structurally-encrypted databases," in *Proc. 24th Int. Conf. Theory Application Cryptol. Inf. Secur.*, Springer, 2018, pp. 149–180.
- [31] R. Poddar, T. Boelter, and R. A. Popa, "Arx: An encrypted database using semantically secure encryption," *Proc. Very Large Data Bases Endowment*, vol. 12, no. 11, pp. 1664–1678, 2019.
- [32] T. Hackenjos, F. Hahn, and F. Kerschbaum, "SAGMA: Secure aggregation grouped by multiple attributes," in *Proc. of the 2020 ACM Int. Conf. Manage. Data*, 2020, pp. 587–601.
- [33] C. Jutla and S. Patranabis, "Efficient searchable symmetric encryption for join queries," in *Proc. 28th Int. Conf. Theory Application Cryptol. Inf. Secur.*, 2022, pp. 304–333.
- [34] E. Anderson, M. Chase, F. B. Durak, K. Laine, and C. Weng, "Precio: Private aggregate measurement via oblivious shuffling," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 1819–1833.
- [35] D. Rathee et al., "CrypTFLOW2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [36] E. Baum et al., "ORQ: Complex analytics on private data with strong security guarantees," in *Proc. ACM Symp. Operating Syst. Princ.*, ACM, 2025, pp. 802–833.
- [37] T. Schneider and M. Zohner, "GMW vs. Yao? efficient secure two-party computation with low depth circuits," in *Proc. Financial Cryptography Data Secur.*, 2013, pp. 275–292.
- [38] V. Kolesnikov, A.-R. Sadeghi, and T. Schneider, "Improved garbled circuit building blocks and applications to auctions and computing minima," in *Proc. 2009 Int. Conf. Cryptol. Netw. Secur.*, 2009, pp. 1–20.
- [39] Y. Lindell, "How to Simulate It—A Tutorial on the Simulation Proof Technique," in *Tutorials on the Foundations of Cryptography*. Berlin, Germany: Springer International Publishing, 2017, pp. 277–346.
- [40] J. Bell et al., "Distributed, private, sparse histograms in the two-server model," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 307–321.
- [41] J. Zhao, K. Xue, Y. Xue, M. Li, B. Zhu, and S. Yuan, "Privacy-preserving statistical analysis with low redundancy over task-relevant microdata," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 4382–4395, 2025.
- [42] C. Dwork and A. Roth et al., "The algorithmic foundations of differential privacy," *Foundations Trends Theor. Comput. Sci.*, vol. 9, no. 3–4, pp. 211–407, 2014.
- [43] "SECRECY: Secure collaborative analytics in untrusted clouds," Accessed: Apr. 2026. [Online]. Available: <http://github.com/CASP-Systems-BU/Secrecy>
- [44] T. P. P. Council, "TPC benchmark H," Accessed: Apr. 2026. [Online]. Available: <https://www.tpc.org/TPC>



Ziwei Peng (Graduate Student Member) received the BS degree in information security from School of the Gifted Young, University of Science and Technology of China (USTC) in July, 2022. He is currently working toward the PhD degree from the School of Cyber Science and Technology, USTC. His research interests include applied cryptography and privacy computing.



Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, USTC, in 2003 and the doctor's degree from the Department of Electronic Engineering and Information Science (EEIS), USTC, in 2007. From 2012 to 2013, he was a postdoctoral researcher with the Department of Electrical and Computer Engineering, University of Florida. He is currently a professor with the School of Cyber Science and Technology, USTC. His research interests include next-generation Internet architecture design, and transmission optimization and network security. He serves on the Editorial Board of several journals, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Wireless Communications*, and *IEEE Transactions on Network and Service Management*. He has served as a Guest Editor for many reputed journals/magazines, and also served as the track/symposium chair for some reputed conferences. He is an IET Fellow.



Jingcheng Zhao (Graduate Student Member, IEEE) received the bachelor's degree from the School of Computer Science and Information Engineering, Hefei University of Technology in 2021. He is currently working toward the PhD degree from the School of Cyber Science and Technology, USTC. His research interests include network security and privacy computing.



Yingjie Xue (Member, IEEE) received the bachelor's degree from the Department of Information Security and the master's degree from the Department of EEIS, USTC, in 2015 and 2018, respectively, and the PhD degree from Computer Science from Brown University in 2023. From 2023 to 2024, she was an assistant professor with the Hong Kong University of Science and Technology (Guangzhou), Financial Technology Thrust. She is currently an associate researcher with the School of Cyber Science and Technology, USTC. Her research interests include privacy computing, blockchain, and network security.



Jinjiang Yang received the BS degree in information security from School of Cyber Science and Technology, USTC in 2021. He is currently working toward the PhD degree from the School of Cyber Science and Technology, USTC. His research interests include network security and applied cryptography.